

TrialCheck

Product Requirements Document

v0.2.0 · Platform-agnostic A/B Experiment Readout Auditor

Version	0.2.0
Status	PRD — PASS
Author	Sidharth Kriplani
Date	May 2025
Language	Python 3.10 / 3.11 / 3.12
Dependencies	Zero — stdlib only
Checks	7 audit checks
Tests	17 unit tests — all passing
Demo scenarios	4 canonical: clean_pass, srm_fail, peeking_warn, guardrail_harm
Claim boundary	Audit helper. Does not run experiments, implement sequential testing, CUPED, or make ship/no-ship decisions.

1. Problem Statement

Most experimentation platforms report a p-value and a lift estimate for the primary metric. Those two numbers are not sufficient to decide whether an experiment result is trustworthy enough to act on.

Before shipping an experiment result, a senior data scientist runs a consistent set of checks:

- **Sample ratio mismatch** — did the assignment pipeline deliver the expected split? An SRM means variant labels may be corrupted.
- **Peeking / early readout** — was the result called before the planned duration, or after multiple interim looks? Conventional p-values are invalid under repeated testing.
- **Practical significance** — is the observed lift large enough to justify a ship decision, independent of statistical significance?
- **MDE context** — is the observed effect at least as large as what the experiment was designed to detect?
- **Continuous metric validity** — for mean-based metrics, is the difference real under an appropriate test that doesn't assume equal variances?
- **Guardrail movement** — did any safety or health metric move in the wrong direction beyond the configured tolerance?
- **Pre-period balance** — were the variants statistically balanced before the experiment started, ruling out assignment bugs or batch confounds?

These checks are well-understood in the experimentation community but are rarely automated. They live in runbooks, reviewer checklists, and institutional memory. Teams skip them under deadline pressure, leading to shipped experiments whose results are not trustworthy.

TrialCheck packages these checks into a single, auditable library call with zero dependencies.

2. Goals and Non-Goals

2.1 Goals

- Accept a structured experiment summary (assignment counts, metric data, guardrails, pre-period covariates) from any source — platform export, warehouse query, or spreadsheet — and return a structured audit report.
- Run 7 audit checks covering the most common experiment-readout failure modes.
- Produce reports in JSON, Markdown, and HTML for consumption by humans and pipelines.
- Zero external dependencies — installable in any Python 3.10+ environment without version conflicts.
- Explicit claim boundary: every report includes a disclaimer that TrialCheck is decision support, not a decision-maker.
- Graceful degradation: missing inputs return `INSUFFICIENT_INPUT` status with recommendations rather than raising exceptions.

2.2 Non-Goals

- Running experiments or assigning users to variants.
- Replacing the source experimentation platform's statistical engine.
- Implementing sequential testing (mSPRT, always-valid p-values, alpha spending).
- Implementing CUPED or any other variance reduction technique.
- Making automatic ship / no-ship decisions.
- Computing prospective power or required sample sizes.
- Multi-arm (A/B/n) experiments in v0.2 — two-arm only.
- Streaming or real-time experiment monitoring.

3. Check Specifications

Each check returns a CheckResult with fields: check (name), status (PASS / WARN / FAIL / INSUFFICIENT_INPUT), detail (human-readable summary), recommendation, and evidence (dict with all computed values for audit trail).

Check	Input Required	Pass Condition	Fail / Warn Condition
SRM	control.n, treatment.n, expected_control_ratio	$p \geq \alpha$ under chi-square df=1	FAIL if $p < \alpha$
Primary Metric (z-test)	control.conversions, treatment.conversions, n	$p < \alpha$ (two-proportion z-test)	WARN if $p \geq \alpha$
Continuous Metric (Welch t)	control.mean, control.std, treatment.mean, treatment.std, n	$p < \alpha$ (Welch t-test)	WARN if $p \geq \alpha$; INSUFFICIENT_INPUT if mean/std absent
Practical Significance	conversions or mean, practical_threshold	$ \text{absolute lift} \geq \text{practical_threshold}$	WARN if $ \text{lift} < \text{threshold}$
MDE Context	conversions or mean, mde	$ \text{absolute lift} \geq \text{mde}$	WARN if $ \text{lift} < \text{mde}$
Peeking Risk	planned_duration_days, actual_duration_days, interim_looks	$\text{actual/planned} \geq 0.70$ AND $\text{interim_looks} == 0$	WARN if ratio < 0.70 or $\text{interim_looks} > 0$
Guardrail Movement	GuardrailMetric: control_value, treatment_value, bad_direction, tolerance	No harmful movement beyond tolerance	FAIL if harmful movement $>$ tolerance
Pre-period Balance (SMD)	PrePeriodCovariate: means, stds per arm	$ \text{SMD} \leq \text{max_abs_smd}$ (default 0.10)	WARN if $ \text{SMD} > \text{max_abs_smd}$

4. Input Model

ExperimentSummary

experiment_id	str — unique identifier for the experiment
metric_name	str — name of the primary metric
control	VariantSummary — assignment counts + optional metric data
treatment	VariantSummary — assignment counts + optional metric data
expected_control_ratio	float — planned randomization split (e.g. 0.5 for 50/50)
alpha	float — significance threshold, default 0.05
planned_duration_days	int None — intended experiment duration
actual_duration_days	int None — actual duration at readout
interim_looks	int — number of interim significance checks, default 0
mde	float None — planned minimum detectable effect (absolute)
practical_threshold	float None — minimum business-meaningful absolute lift
guardrails	list[GuardrailMetric] None — optional safety/health metrics
pre_period_covariates	list[PrePeriodCovariate] None — optional balance covariates

VariantSummary

name	str — variant label (e.g. 'control', 'treatment')
n	int — assignment count
conversions	int None — conversion count (binary metric path)
mean	float None — metric mean (continuous metric path)
std	float None — metric standard deviation (continuous metric path)

GuardrailMetric

name	str — guardrail metric name
control_value	float — observed metric value in control
treatment_value	float — observed metric value in treatment

bad_direction	'increase' 'decrease' — which movement is harmful
tolerance	float — maximum acceptable harmful movement
unit	str None — unit label for display

5. Output Model

TrialReport

experiment_id	str — echoed from input
metric_name	str — echoed from input
overall_status	CheckStatus — FAIL > WARN > INSUFFICIENT_INPUT > PASS rollup
checks	list[CheckResult] — one entry per check run
interpretation	str — human-readable overall readout summary
metadata	dict — tool version, claim boundary

CheckResult

check	str — check name
status	PASS WARN FAIL INSUFFICIENT_INPUT
detail	str — computed values and what was observed
recommendation	str — what to do given this status
evidence	dict None — all computed values for audit trail (p-values, lifts, etc.)

Overall Status Rollup

The overall status is determined by the worst individual check status using the ordering: FAIL > WARN > INSUFFICIENT_INPUT > PASS. A single FAIL check causes the overall report to be FAIL regardless of all other checks. A report with all PASS and some INSUFFICIENT_INPUT rolls up to WARN, signaling that the audit is incomplete.

6. Demo Scenarios

TrialCheck ships four pre-built canonical scenarios that demonstrate each major failure mode. All scenarios are based on the same synthetic checkout experiment (control n=15,000, treatment n=15,000, 90-day checkout conversion rate measurement).

Scenario	Overall	Primary Metric	What Fires	Purpose
clean_pass	PASS	Significant (p < 0.001)	No check fires. All data supplied.	Shows a complete, ship-ready readout.
srm_fail	FAIL	Not interpretable	SRM: observed 56/44 split vs 50/50 planned. chi-square p < 0.001.	Shows why SRM blocks all other interpretation.
peeking_warn	WARN	Appears significant	Peeking: 36% of planned duration, 2 interim looks.	Shows why early readouts are unreliable.
guardrail_harm	FAIL	Significant	Guardrails: revenue/user -4.4% (exceeds 5% tolerance), refund rate +0.018 (exceeds 0.005 tolerance).	Shows why primary metric alone is not enough.

7. Success Criteria

Criterion	Target	Status
All 7 checks implemented and tested	17/17 unit tests passing	PASS
Zero external dependencies	pip install trialcheck on clean Python 3.10+	PASS
Four canonical demo scenarios	clean_pass=PASS, srm_fail=FAIL, peeking_warn=WARN, guardrail_harm=FAIL	PASS
Graceful degradation on missing input	INSUFFICIENT_INPUT returned, no exceptions	PASS
Explicit claim boundary in every report	metadata.claim_boundary field present	PASS
CI passing on Python 3.10 / 3.11 / 3.12	GitHub Actions matrix	PASS
Report outputs: JSON, Markdown, HTML	write_report() with all three formats	PASS
Statistical correctness: SRM	chi-square df=1 via erfc — exact	PASS
Statistical correctness: z-test	pooled SE under H0 — correct for null testing	PASS
Statistical correctness: Welch t-test	Welch-Satterthwaite dof, betainc / erfc fallback	PASS
Statistical correctness: SMD	pooled SD formulation — textbook	PASS
MDE check framing	observed vs planned MDE only — no post-hoc power	PASS

8. Claim Boundary

TrialCheck is explicit and honest about what it does not do. Every TrialReport includes a `claim_boundary` field in its metadata, and the README, METHODOLOGY.md, and Interview Defense document all repeat this boundary.

- Does not run experiments or assign users
- Does not replace the source experimentation platform's statistical engine
- Does not implement sequential testing — a conventional p-value after peeking is not valid, and TrialCheck does not correct for this
- Does not implement CUPED or variance reduction
- Does not compute prospective power or required sample sizes
- Does not make automatic ship/no-ship decisions
- Does not prove causal validity
- The MDE context check is not a retrospective power calculation — it compares observed lift against the planned MDE and nothing more

Resume-Safe Claim

Built TrialCheck, a platform-agnostic Python library (zero dependencies, 17 tests, CI on Python 3.10/3.11/3.12) that audits completed A/B experiment readouts against seven checks: sample-ratio mismatch (chi-square $df=1$), primary metric two-proportion z-test, continuous metric Welch t-test (Welch-Satterthwaite dof), practical significance, MDE context, peeking risk, guardrail movement, and pre-period covariate balance (SMD). Produces structured JSON/Markdown/HTML reports with per-check PASS/WARN/FAIL/INSUFFICIENT_INPUT status, explicit claim boundary, and four canonical demo scenarios covering clean-pass through guardrail-harm readouts.